

EGCO 221 – Group Project 1 (Reversing Marbles)

The project can be done in a group of ≤ 5 students. Each group must do the project by themselves:

- **Everyone involved in cheating, either as a source or copier, will get ZERO point.**
- If late submitting group copies code from a graded group, the graded group will still be penalized.
- **In case of AI usage, you must write your own prompts and perform your own acquisition with the AI.** Due to the dynamic nature of generative AI, it is very unlikely that any 2 groups will get identical generated code even when using identical prompts.
 - Therefore, submitting identical code will be counted as cheating.
 - Don't use generated content obtained by other groups as your own.
 - Don't share generated content you get from the AI with other groups.
 - If any suspicious arises, I may ask both groups to show their chat history with the AI. Failure to do so will result in cheating penalty.
 - Add acknowledgment of AI usage at the end of your report (see 3 and 4.6)

1. Given a board of size $2n+1$ that contains n white marbles and n black marbles separated by a space, our purpose is to reverse this board. Conditions to move any marble are as follows.

1.1 White marble can only go right

- It can move to RHS space, i.e. from "w _" to "_w"
- It can jump over a black marble to RHS space, i.e. from "w b _" to "_ b w"

1.2 Black marble can only go left

- It can move to LHS space, i.e. from "_ b" to "b _"
- It can jump over a white marble to LHS space, i.e. from "_ w b" to "b w _"

2. **Programing**: the program must be written in Java.

- 2.1 Ask user for n which is #white and #black marble; n must be at least 2. Create a board of $2n+1$ marbles. All marbles must be labelled with IDs e.g. w0, w1, b0, b1.
- 2.2 Keep asking user for marble ID. Check whether the chosen marble is movable. If it is, then move it. At any time, the user must be able to switch to auto mode.
- 2.3 In auto mode, find a solution (if there is one) starting from the switching state in (2.2).
- 2.4 The program must be able to loop for a new game with different board size.
- 2.5 The program must be able to check user input and handle invalid input without crashing.
- 2.6 Your source file (.java) must be in folder Project1_XXX where XXX = full ID of the group representative, assuming that this folder is under Maven's "src/main/java/" structure. The first line of all source files must be comments containing names and IDs of all members.

3. You are required to use backtracking with either explicit or implicit stack to solve this puzzle. Recursive methods are counted as using (runtime) stack implicitly.

- You can search and use any algorithm or even pieces of code, provided that the sources are properly acknowledged.
- These sources must be from your own research e.g. paper, textbook, Internet, AI tools, etc.
- Using classmates as references is considered cheating.

```

--- exec-maven-plugin:3.0.0:exec (default-cli) @ javasolutions ---
Enter number of white marbles = 2
Initial  >> w0 w1 __ b0 b1
Step 1  >> Enter marble ID or A to switch to auto mode = w0
          Cannot move w0
Step 2  >> Enter marble ID or A to switch to auto mode = b1
          Cannot move b1
Step 3  >> Enter marble ID or A to switch to auto mode = w1
          Move right
          w0 __ w1 b0 b1
Step 4  >> Enter marble ID or A to switch to auto mode = b0
          Jump left
          w0 b0 w1 __ b1
Step 5  >> Enter marble ID or A to switch to auto mode = b1
          Move left
          w0 b0 w1 b1 __
Step 6  >> Enter marble ID or A to switch to auto mode = w1
          Jump right
          w0 b0 __ b1 w1
Step 7  >> Enter marble ID or A to switch to auto mode = b0
          Cannot move b0
Step 8  >> Enter marble ID or A to switch to auto mode = w0
          Jump right
          __ b0 w0 b1 w1
Step 9  >> Enter marble ID or A to switch to auto mode = b0
          Move left
          b0 __ w0 b1 w1
Step 10 >> Enter marble ID or A to switch to auto mode = b1
          Jump left
          b0 b1 w0 __ w1
Step 11 >> Enter marble ID or A to switch to auto mode = w0
          Move right
          b0 b1 __ w0 w1

Done !!

```

```

--- exec-maven-plugin:3.0.0:exec (default-cli) @ javasolutions ---
Enter number of white marbles = 2
Initial  >> w0 w1 __ b0 b1
Step 1  >> Enter marble ID or A to switch to auto mode = w1
          Move right
          w0 __ w1 b0 b1
Step 2  >> Enter marble ID or A to switch to auto mode = b0
          Jump left
          w0 b0 w1 __ b1
Step 3  >> Enter marble ID or A to switch to auto mode = A

Auto 1  >> w0 b0 w1 b1 __
Auto 2  >> w0 b0 __ b1 w1
Auto 3  >> __ b0 w0 b1 w1
Auto 4  >> b0 __ w0 b1 w1
Auto 5  >> b0 b1 w0 __ w1
Auto 6  >> b0 b1 __ w0 w1

Done !!

```

Design a better user interface by yourself. Good user interface doesn't mean fancy output, but rather how your program is easy to understand & to use even without user manual. For example,

- Can user continue playing without having to restart the program?
- Are the instructions clear enough?
- Do you warn about valid input and/or handle invalid input?
- Is the output properly formatted and understandable?

```
--- exec-maven-plugin:3.0.0:exec (default-cli) @ javasolutions ---
Enter number of white marbles = 2
Initial >> w0 w1 __ b0 b1
Step 1 >> Enter marble ID or A to switch to auto mode = w1
          Move right
          w0 __ w1 b0 b1
Step 2 >> Enter marble ID or A to switch to auto mode = b0
          Jump left
          w0 b0 w1 __ b1
Step 3 >> Enter marble ID or A to switch to auto mode = w1
          Move right
          w0 b0 __ w1 b1
Step 4 >> Enter marble ID or A to switch to auto mode = a
No solution !!
```

4. **Report in THAI, with page numbers:** describe at least the following

4.1 Short user manual

4.2 **Stack (including ArrayDeque, runtime stack) for backtracking** + other data structures

- Explain how stack is used in your backtracking algorithm. Give the name of your (explicit) stack, locate it in your program (capture code section with your stack, explain which class/method it is a member of), and what values are kept in it.
 - Backtracking must always work with stack. Even if you use recursive method, you still have to explain the above points. Do further research on how runtime stack works with recursive method and give explanation in the context of reversing-marbles solution.
 - Stacks used in trivial tasks, that don't help the algorithm going forward or backward (see 4.3), will be counted only as other data structures in the next bullet.
- Other data structures e.g. ArrayList, LinkedList, etc. Be specific about each data structure object: give the name of the object, locate it in your program, explain reasons for using it and what values are kept in it.

4.3 **Backtracking algorithm** to find at least 1 solution. Source code listing without any explanation isn't counted as a proper report. Explain the following points:

- Forwarding steps
 - The starting point and search direction e.g. from first/left to last/right, from last to first, etc.
 - The calculation in each step + values added to or removed from your stack in each step.
- Backtracking steps
 - Conditions that trigger backtracking.
 - The calculation in each step + values added to or removed from your stack in each step.
- How the algorithm concludes that there is no solution?

4.4 Demonstrate how your backtracking works until a solution is found or not found, using the following initial states:

- Initial state (1) w0 w1 __ b0 b1
- Initial state (2) w0 b0 __ w1 b1

For each initial state, show the board & stack content in every forwarding step and every backtracking step. Don't conjure up the moves. Your demo must match what your program actually does and your explanation in 4.3. Note that there can be ≥ 1 solution to the puzzle. Yours may or may not be the same as the given example.

4.5 Any limitation of your program. Convenient limitation isn't counted as proper limitation. For example, don't give a limitation that your program will crash if n is negative just because you are too lazy to check for valid input.

4.6 References or declaration of originality

- If you design the data structures/algorithms and write the whole program by yourself, explicitly declare that everything is done by yourself. But if it is found later that this is not true, it will be counted as cheating.
- Otherwise, valid references (in proper IEEE or APA) and URLs must be given. The URLs must point to the code you take from and can be accessed without login.
- A report without any reference or declaration of originality will get point deduction.
- For AI usage, add the acknowledgment of AI usage at the end of the report.

Grading

Programming (10 points)

- 2 points Minimum requirements + correct solution for fully auto mode
- 2 points Correct solution (or no solution) for any initial state
- 2 points User interface + exception handling
- This part must be your own effort. Even if 2 groups take algorithms or pieces of code from the same source, it would be impossible to have the same/similar coding for the user interface. Therefore, same (or too similar) user interface coding is considered cheating
- 4 points Programming + Java Collection APIs
- Although this is not a programming course, your program should still be well structured
 - Even if you use pieces of code from the Internet. Convert it to proper OOP with Java collection APIs

Report (10 points)

- 3 points Stack for backtracking + other data structures
- 5 points Backtracking algorithm + demos for 2 initial states
- 2 points Manual, limitation, reference/declaration of originality, others (writing, format, etc.)

Submission

1. A report in only 1 PDF file. The cover page must contain names & IDs of everyone in your group. All pages except the cover page must have page numbers.
2. Source files (*.java)
3. File readme.txt containing names & IDs of everyone in your group
4. Put all files in folder Project1_XXX (see 2.6) and zip the folder
 - The group representative submits the whole project to Google Classroom
 - The other group members submit only readme.txt to Google Classroom

Late submission

- -0.5 point for <1 week late
- -1 point for each 1 full week late